PERSISTENCIA

Objeto que puede almacenar su estado para ser reusado.

Introducción

Almacenar

- Base de Datos
- Tablas
- Claves primarias
- Índices
- Relaciones entre tablas
- Formas normales

OO

- Objetos: encapsula comportamiento y estado
- Herencia
- Colecciones
- Clases: abstractas, concretas

Forma de persistencia

- JDBC (Java database **Connectivity**): API estándar para acceder a BD relacional.
- ORM (Object-Relational **Mapping**): delega el acceso a la BD a una herramienta externa. Hibernate, TopLink Java Data Object.
- JPA (Java Persistence **API**): integra el modelo relacional y orientado-objetos. Incluida en Java EE 6.

JPA
especificaciones

- ORM: Mecanismo que mapea objetos a datos almacenados en una BD.
- API para manejar entidades: para realizar operaciones sobre la BD (crear, leer, etc.)
- Lenguaje de consulta (JPQL): recuperar datos con un lenguaje query OO.
- Mecanismo de transacciones y bloqueo que controla accesos concurrentes a los datos (JTA API).

//Conceptos previos

a) Entidad

b) Mapeo relacional-objetos
   - Anotaciones
   - Descriptores XML

c) Unidad de persistencia

a) Entidad

Objetos → Son instancias que viven en memoria.

Entidad → Son objetos que viven en memoria y persisten en una bd.

Especificadas por JPA.

b) Mapeo relacional-objetos

- Anotaciones: Proveen información sobre clases, métodos, variables o cualquier otro elemento del programa.
  - Ej. @Override

  El mapeo esta definido en **jakarta.persistence**

- Descriptores XML: El mapeo esta definido en un archivo externo xml y sera desplegado con la entidad.

c) Unidad de persistencia

- Se genera un archivo persistence.xml
- Contiene toda la información necesaria para conectarse a la bd y para determinar el modo de generar la bd.
- El proveedor, en nuestro caso, **eclipseLink**

Recursos:

1. NetBeans -> **Actual**
2. JDK -> **Versión 11**
3. Servidror -> **GlassFish Server 7.0.15**

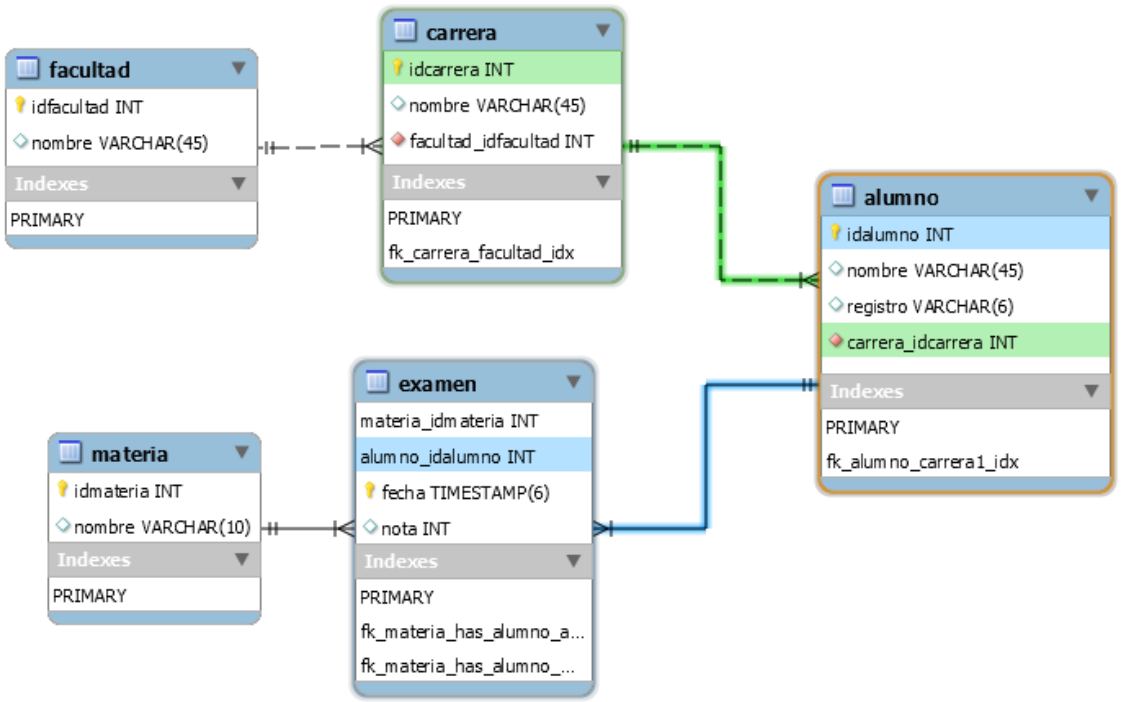
App Java Enterprise

Incluye:

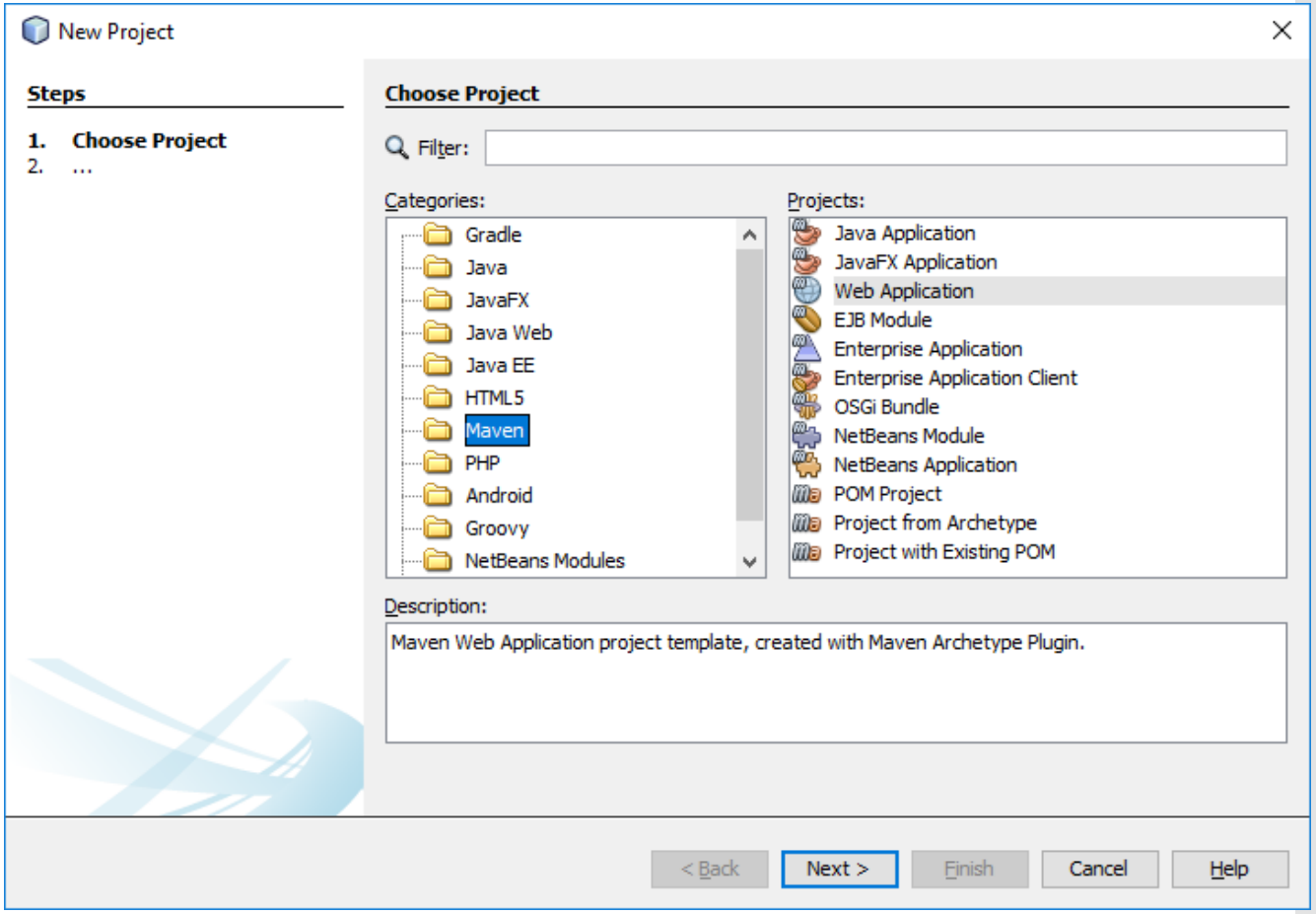
1. **Modelo de Datos. Crear la BD**
2. **Crear proyecto Web. Diseñar la interfaz**
3. **Conexión a MySQL**
4. **Generación de Entidades**
5. **Generación de Sesiones Beans**
6. **Diseño con patrones,** Model-View-Controller (MVC) y Session Facade

1.Crear bd MySql

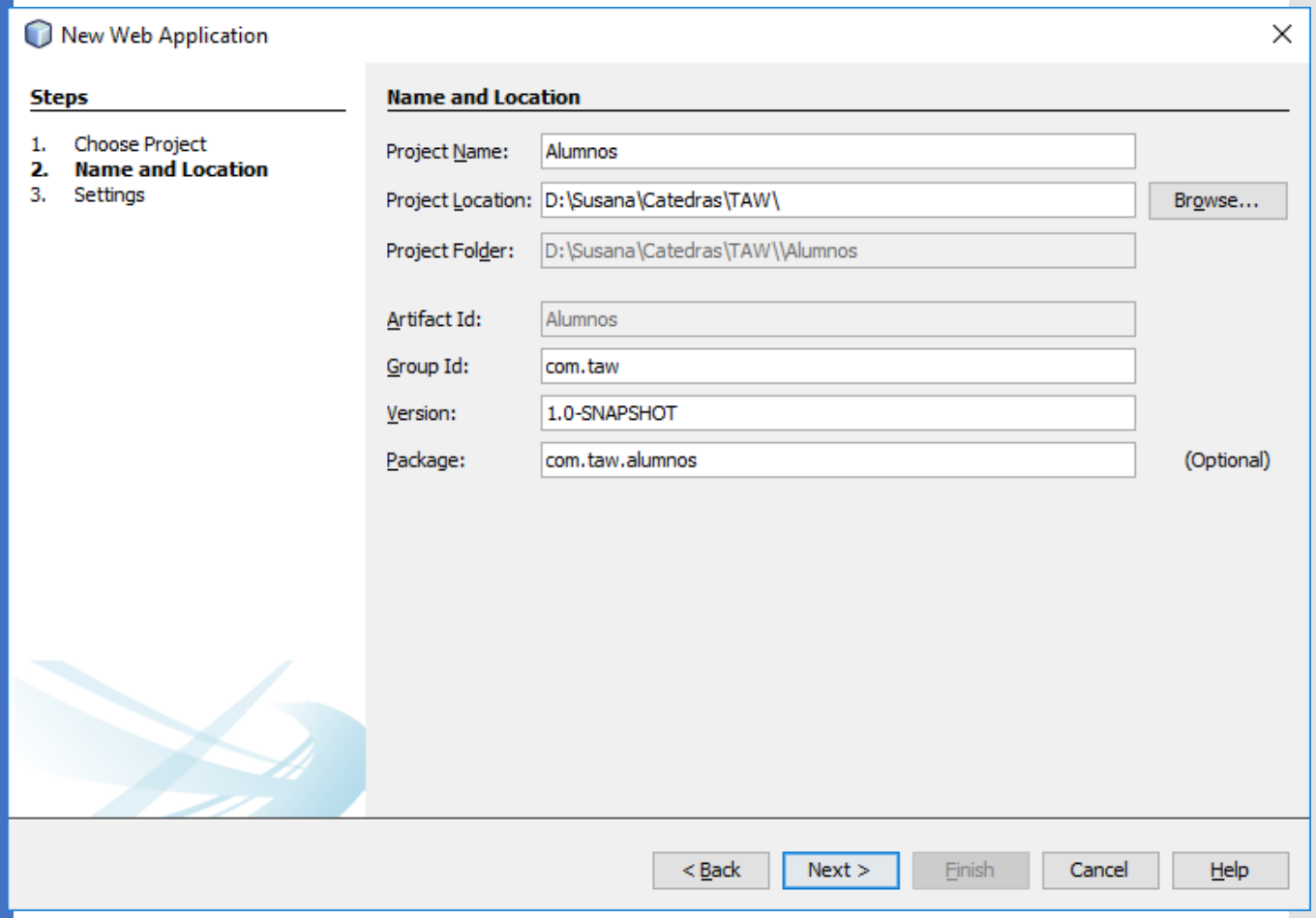
Sistema de gestión alumnos:

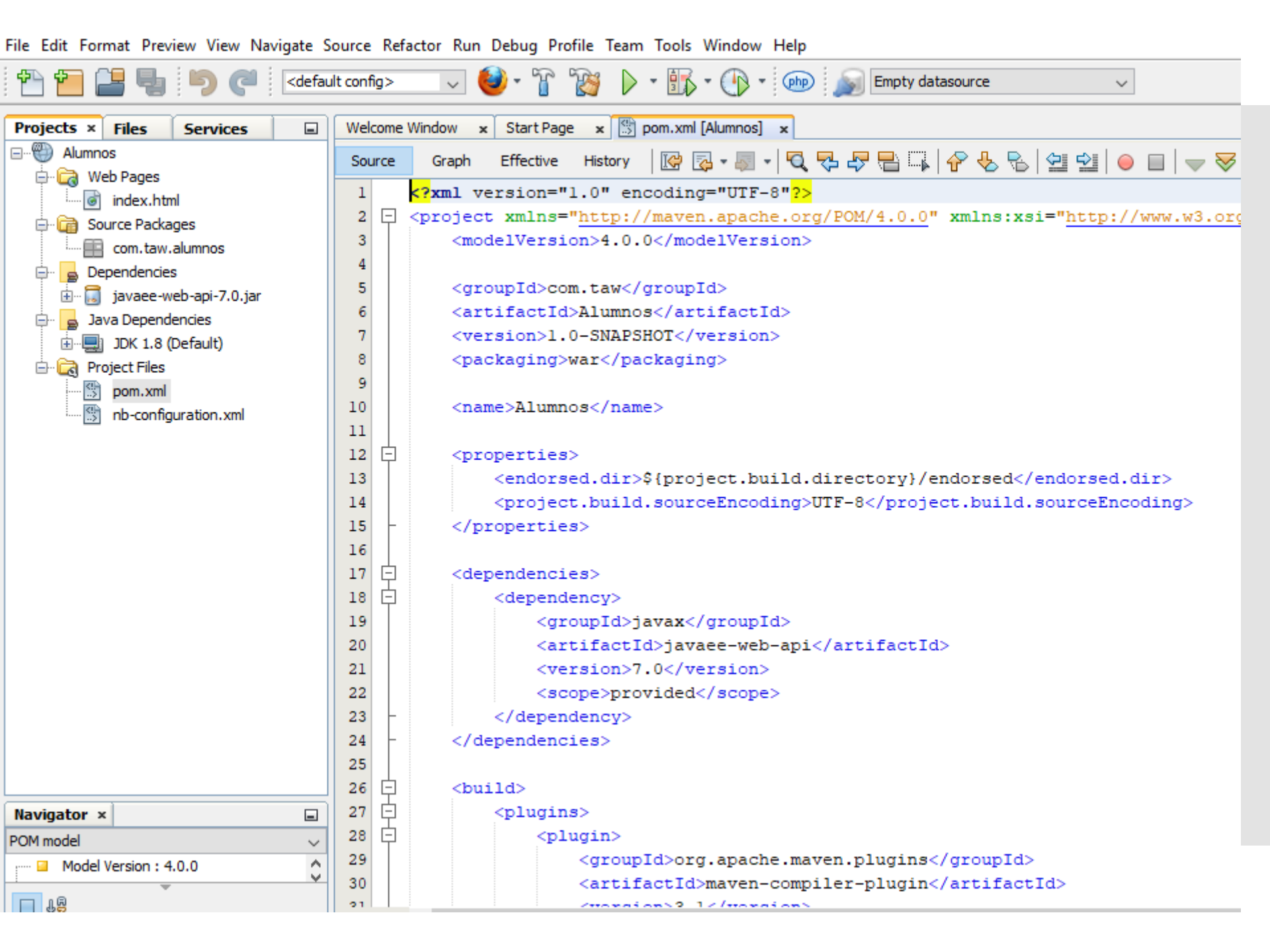


2. Crear proyecto con maven (propuesta)

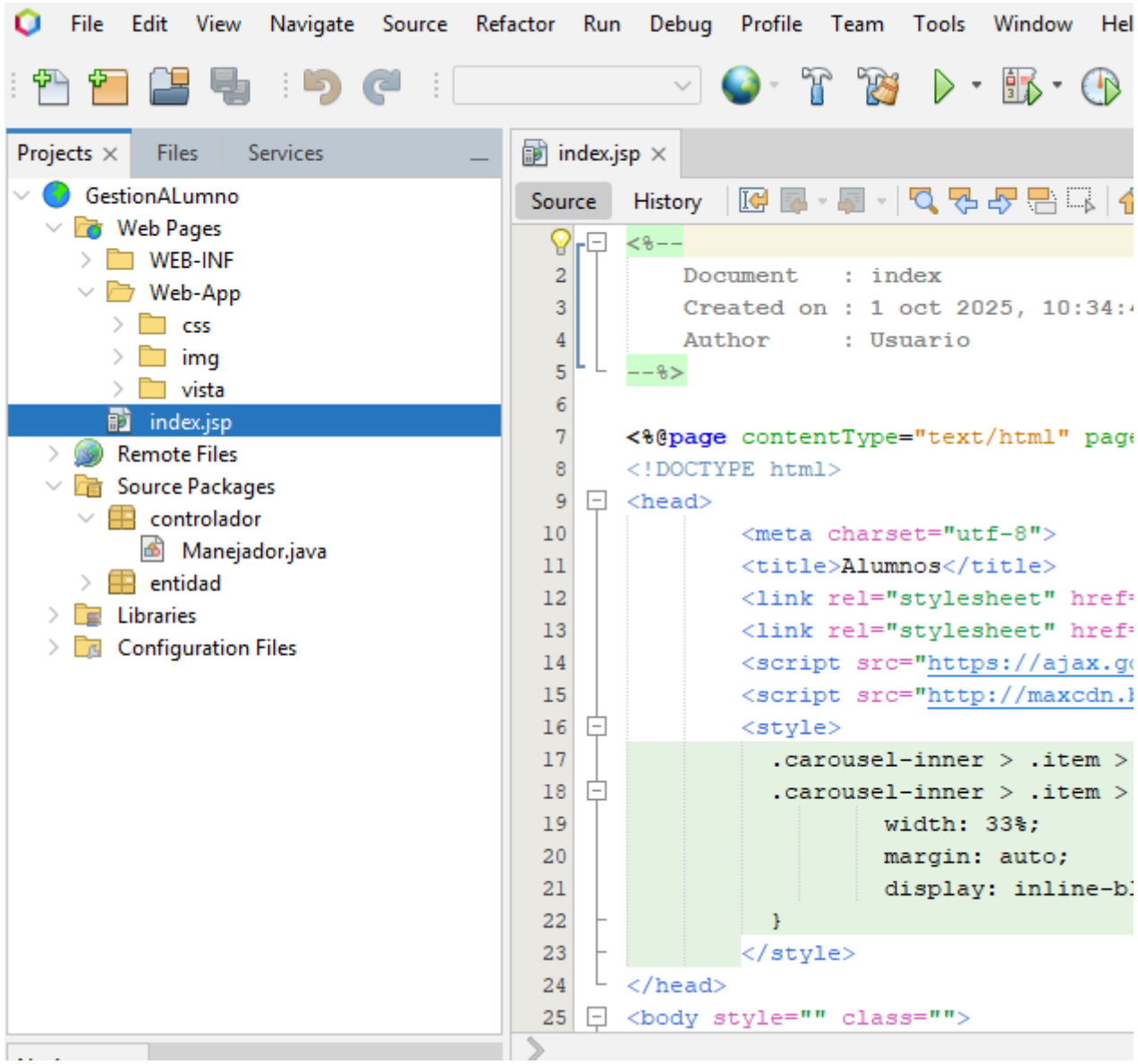


http://maven.apache.org/


New Web Application
Steps
1. Choose Project
2. Name and Location
3. Settings
Name and Location
Project Name: Alumnos
Project Location: D:\Susana\Catedras\TAW\
Browse...
Project Folder: D:\Susana\Catedras\TAW\Alumnos
Artifact Id: Alumnos
Group Id: com.taw
Version: 1.0-SNAPSHOT
Package: com.taw.alumnos
(Optional)
< Back
Next >
Finish
Cancel
Help


File Edit Format Preview View Navigate Source Refactor Run Debug Profile Team Tools Window Help
<default config>
Empty datasource
Projects
Files
Services
Alumnos
Web Pages
index.html
Source Packages
com.taw.alumnos
Dependencies
javaee-web-api-7.0.jar
Java Dependencies
JDK 1.8 (Default)
Project Files
pom.xml
nb-configuration.xml
Navigator
POM model
Model Version : 4.0.0
Welcome Window
Start Page
pom.xml [Alumnos]
Source
Graph
Effective
History
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org
<modelVersion>4.0.0</modelVersion>
<groupId>com.taw</groupId>
<artifactId>Alumnos</artifactId>
<version>1.0-SNAPSHOT</version>
<packaging>war</packaging>
<name>Alumnos</name>
<properties>
<endorsed.dir>${project.build.directory}/endorsed</endorsed.dir>
<project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
</properties>
<dependencies>
<dependency>
<groupId>javax</groupId>
<artifactId>javaee-web-api</artifactId>
<version>7.0</version>
<scope>provided</scope>
</dependency>
</dependencies>
<build>
<plugins>
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-compiler-plugin</artifactId>

Agregar interfaz



Práctico:


localhost:8080/AppAlumnos/
SISTEMA
ALUMNOS
Inicio
Facultades
Docentes
Alumnos
Agregar
Listar
Java
JAVA
Java
Web Services
Desarrollo Backend

3. Crear la unidad de Persistencia.

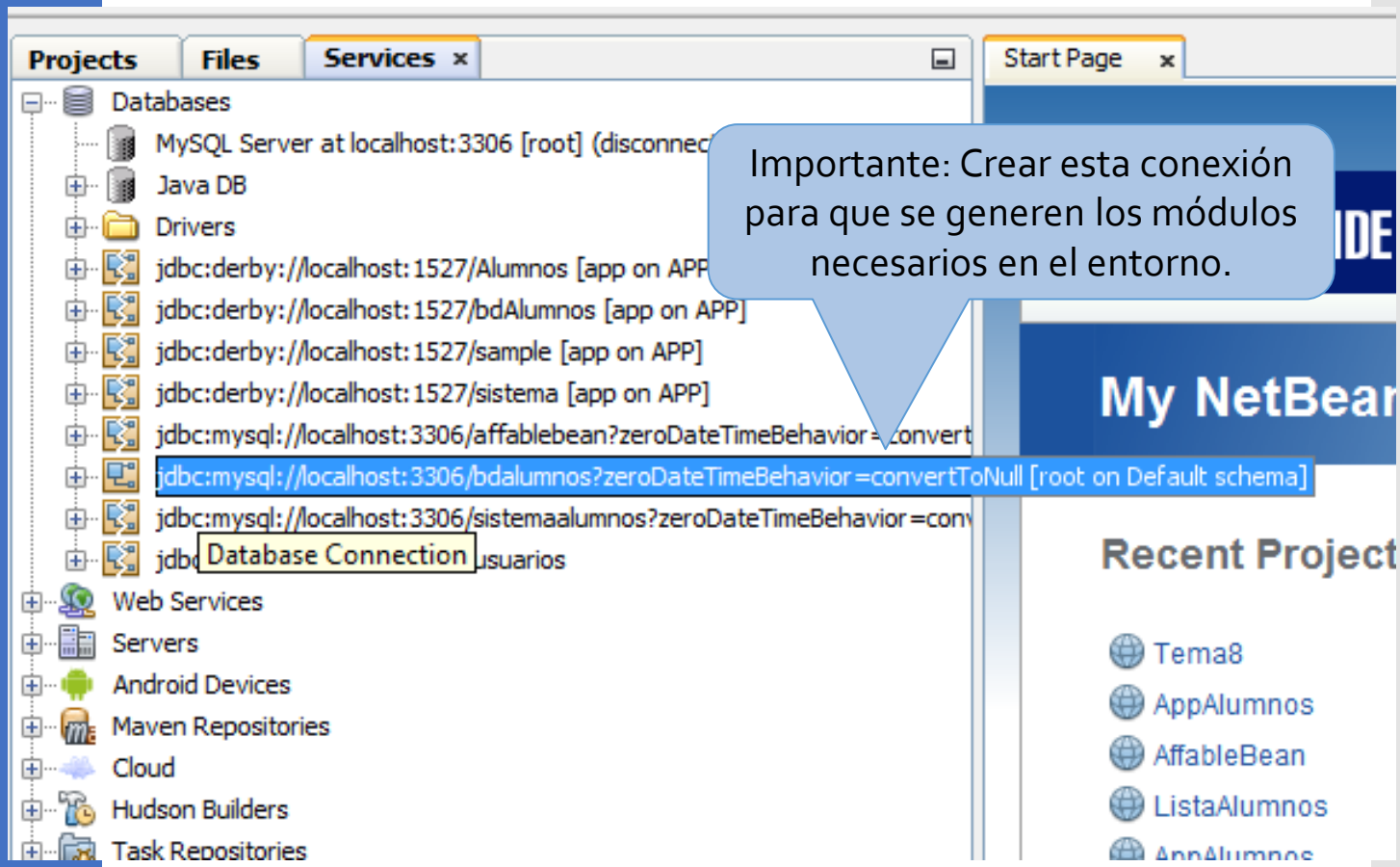
Conexión a MySql sobre GlassFish

Esta comunicación se hace posible con la API de Java Database Connectivity (**JDBC**). Es una biblioteca de integración contenida en el JDK, requerida para comunicar SQL y Java.

Se crea un **pool** de conexiones en el servidor GlassFish. Para que el servidor se comunique directamente con la base de datos, se requiere convertir las llamadas JDBC directamente en un protocolo específico de MySQL.

Un **data source** permite a las aplicaciones la conexión a una bd, que se obtiene del pool de conexiones, buscando en el directorio de nombres (JNDI).

(… crear la conexión jdbc:mysql)



Crear conexión Pool y DataSource

Existen dos caminos:

1. Consola de GlassFish: se introducen manualmente los detalles de conexión de bd (usuario, contraseña y URL).



2. IDE de NetBeans: extrae detalles de conexión directamente de la conexión bd existente, eliminando así posibles problemas de conectividad.



https://dev.mysql.com/downloads/connector/j/?os=26

1. Consola de GlassFish

1. Desde el IDE **iniciar** el servidor GlassFish.
2. Abrir **View Domain Admin Console**

   [user: admin pass: admadmin]
3. Expandimos Resourses > JDBC y click **JDBC Conection pools**
4. Click **NEW** y completamos:

   Name: BDAlumnosPool

   Resource Type: javax.sql.ConnectionPoolDataSource

   Database Vendor: MySql Next
5. Finish

6. Click en el pool creado.
7. Cambiar a:

Datasource Classname:

MySQL 5: com.mysql.jdbc.jdbc2.optional.MysqlDataSource

MySQL 8: com.mysql.cj.jdbc.MysqlDataSource

8. Save
9. Click en tab **Additional Properties** y agregar las propiedades:

user: root

password: .....

useSSL: **false**

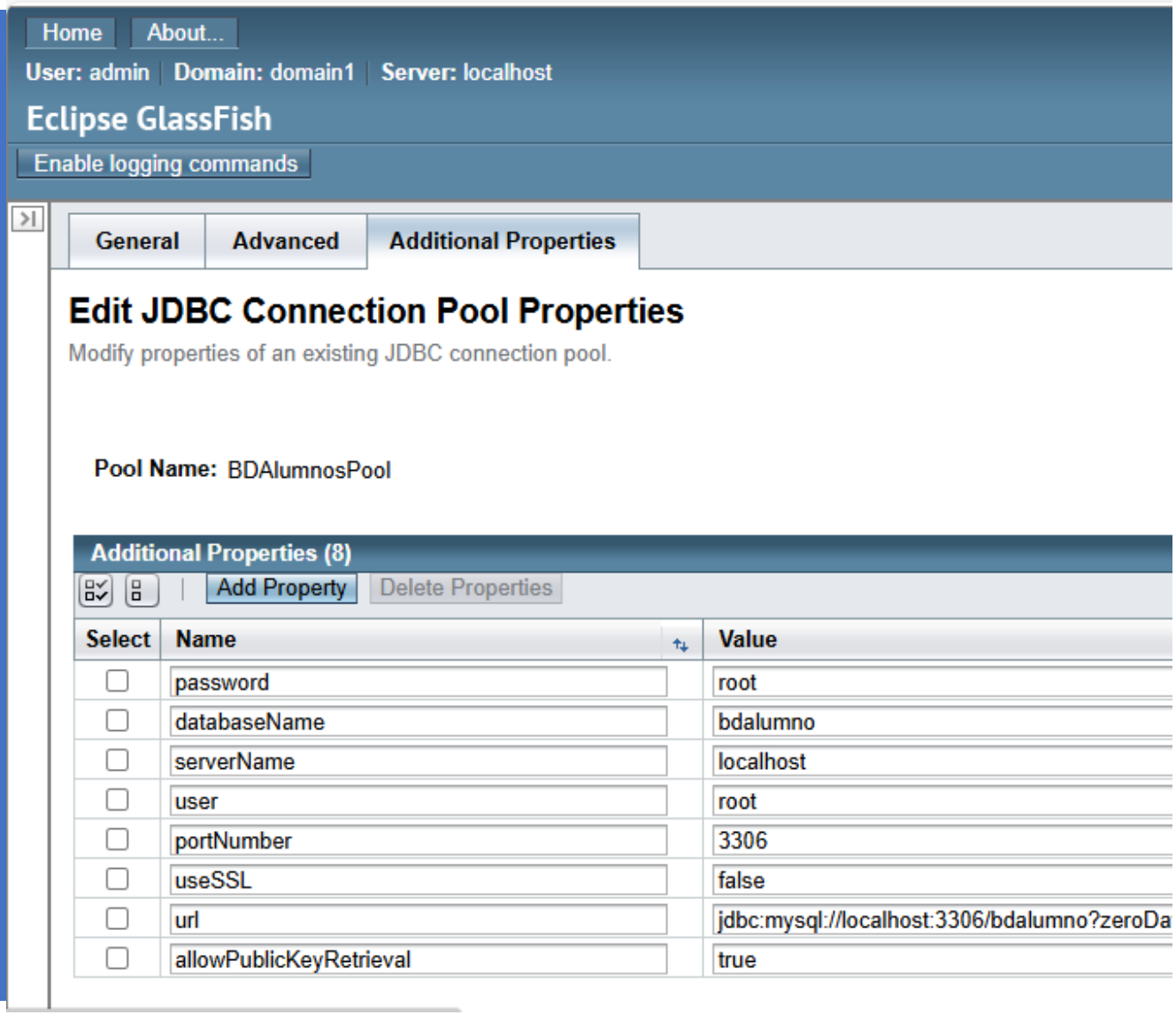
url:

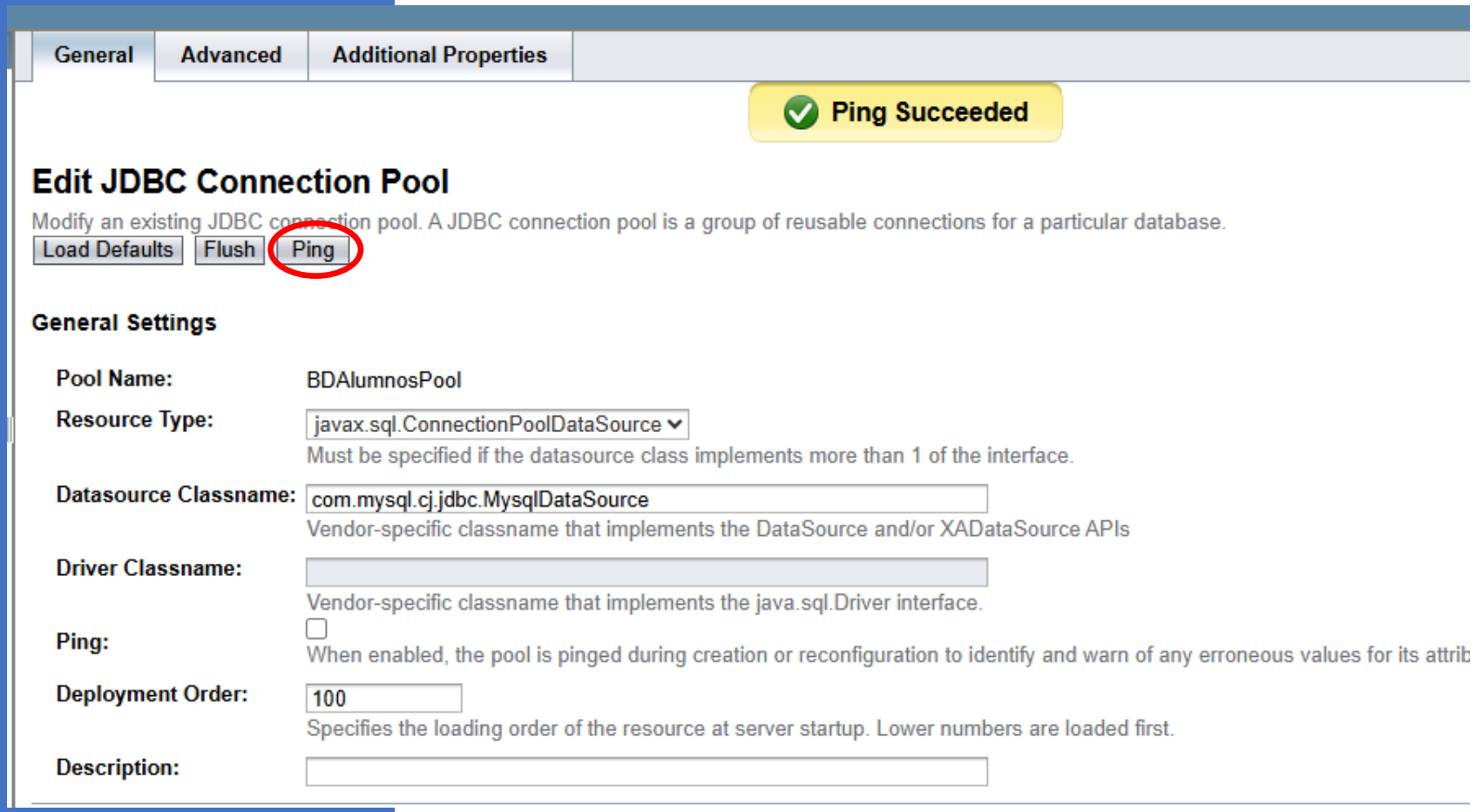
bdalumnos?zeroDateTimeBehavior=ConvertToNull&serverTimezone=UTC

Nota: Prestar atención al **url** y respetar mayúsculas

10. Save
11. Click TAB **General** y luego click **Ping. Deberias ver msg Ping Exitoso**

Ejemplo de propiedades




General
Advanced
Additional Properties
Ping Succeeded
Edit JDBC Connection Pool
Modify an existing JDBC connection pool. A JDBC connection pool is a group of reusable connections for a particular database.
Load Defaults
Flush
Ping
General Settings
Pool Name:
BDAlumnosPool
Resource Type:
javax.sql.ConnectionPoolDataSource
Must be specified if the datasource class implements more than 1 of the interface.
Datasource Classname:
com.mysql.cj.jdbc.MysqlDataSource
Vendor-specific classname that implements the DataSource and/or XADataSource APIs
Driver Classname:
Vendor-specific classname that implements the java.sql.Driver interface.
Ping:
When enabled, the pool is pinged during creation or reconfiguration to identify and warn of any erroneous values for its attrib
Deployment Order:
100
Specifies the loading order of the resource at server startup. Lower numbers are loaded first.
Description:

12. Click **JDBC Resources**
13. Click **NEW** y completamos:
    JNDI Name: jdbc/bdalumnos
    Connection Pool: BDAlumnosPool
14. OK

Otra forma 2. IDE de Netbeans

1. Crear un nuevo archivo en el menu principal del IDE.
2. Elegir categoria **GlassFish** y luego seleccionar **JDBC Resource**
3. Seguir el asistente:
   1. Tildar **Create New JDBC Connection Pool**
   2. Setear el data source:

      JNDI Name: jdbc/bdalumnos // POR CONVENCION DE NOMBRE JNDI

      Object Type: user

      Enabled: true //Next
4. //Next
5. Elegir nombre del pool BDAlumnosPool.

   Tildar **Extract from Existing Connection**, y elegir

   jdbc:mysql://localhost:3306/bdalumnos. //Next
6. Seleccionar:

   Datasource Classname:

   MySQL 5: com.mysql.jdbc.jdbc2.optional.MysqlDataSource
   MySQL 8: com.mysql.cj.jdbc.MysqlDataSource

   Resource Type: javax.sql.ConnectionPoolDataSource

   User: ....

   Password:... //Finish

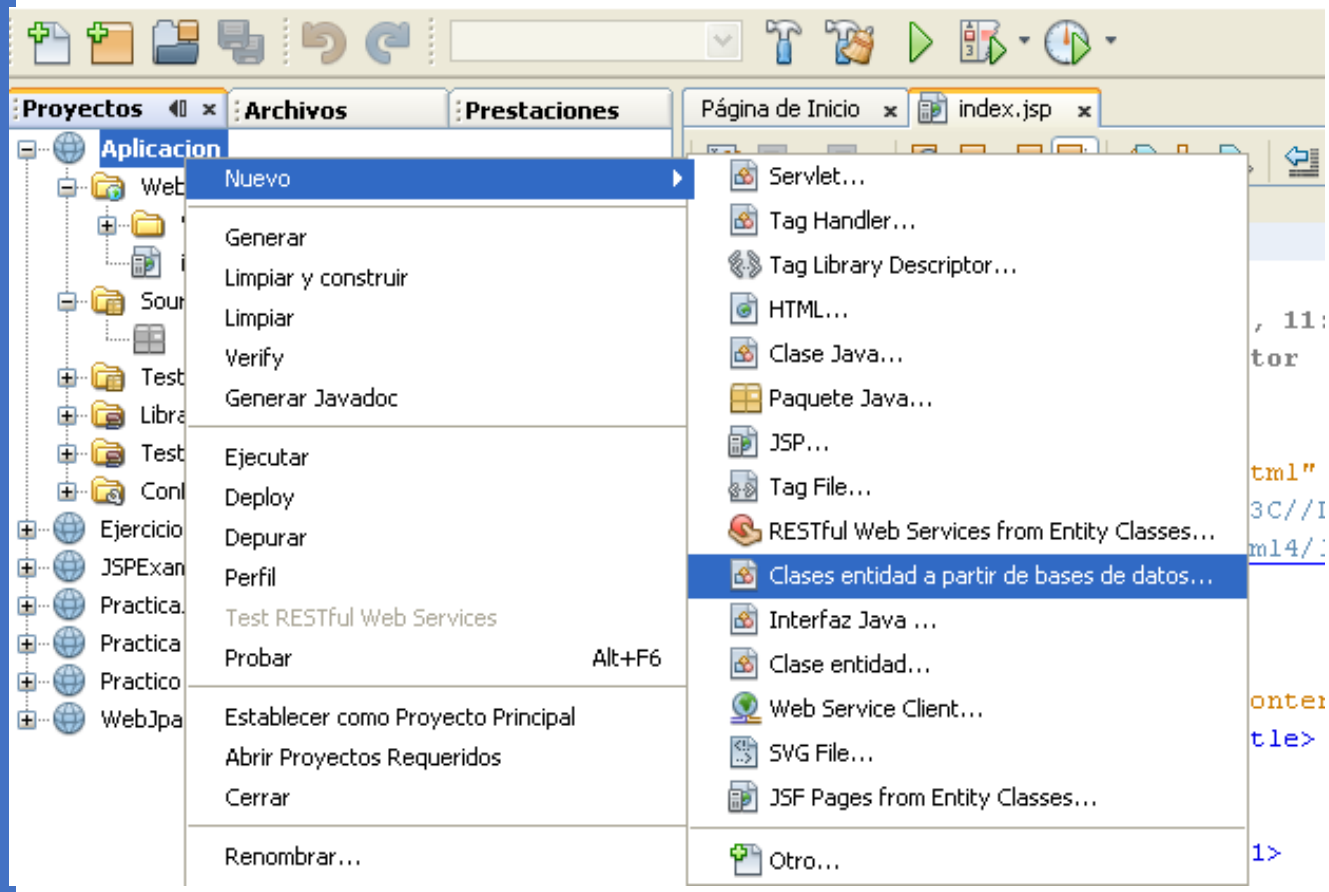
6. Revisar el archivo glassfish-resources.xml de tu proyecto.

App Java Enterprise

Incluye:

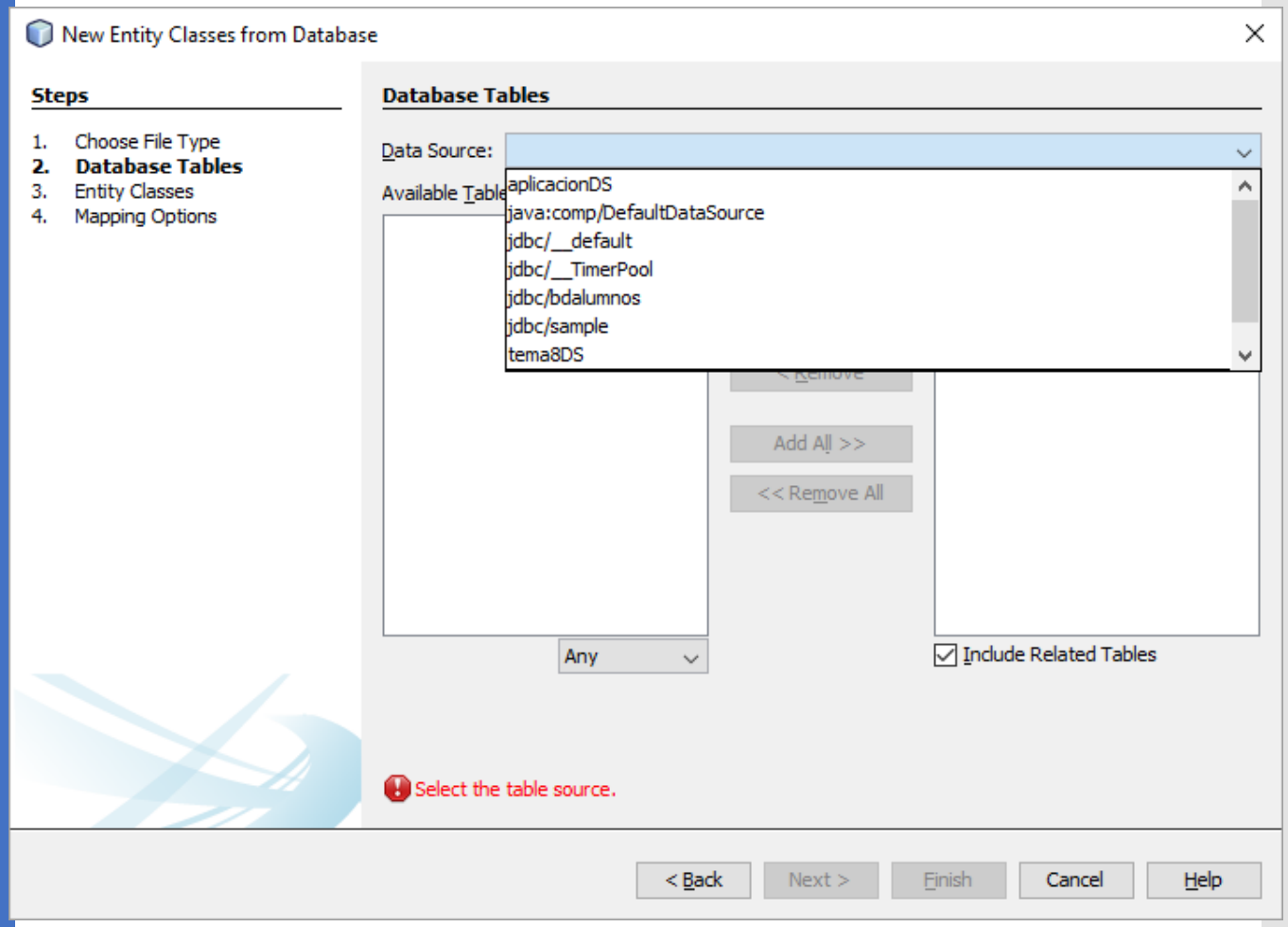
- Proyecto de una Aplicación Web
- Diseño de la interfaz de usuario
- Modelo de Datos
- Conexión a MySQL
- **Generación de Entidades**
- Generación de Sesiones Beans
- Diseño con patrones, Model-View-Controller (MVC) y Session Facade

4. Generar la entidad

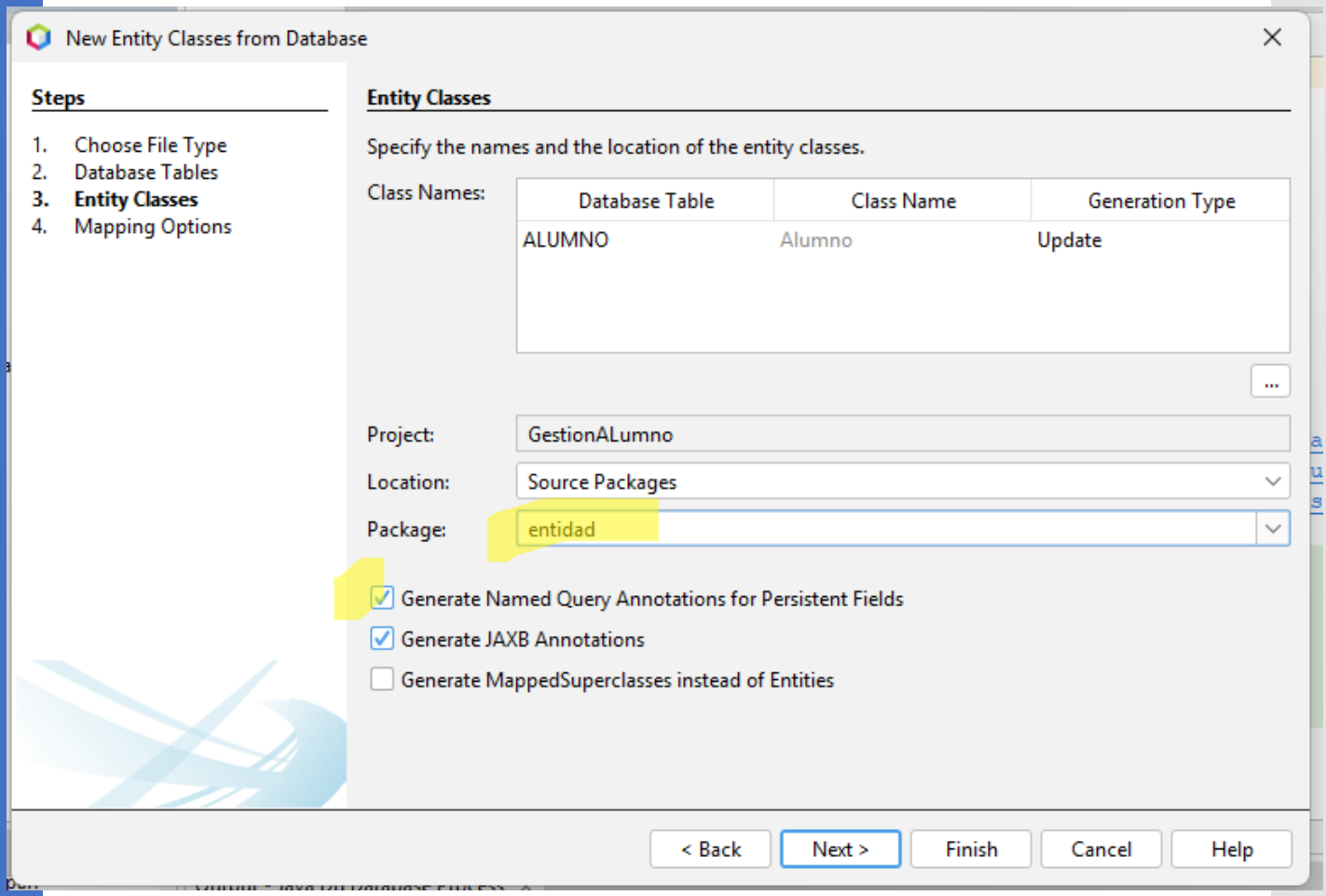


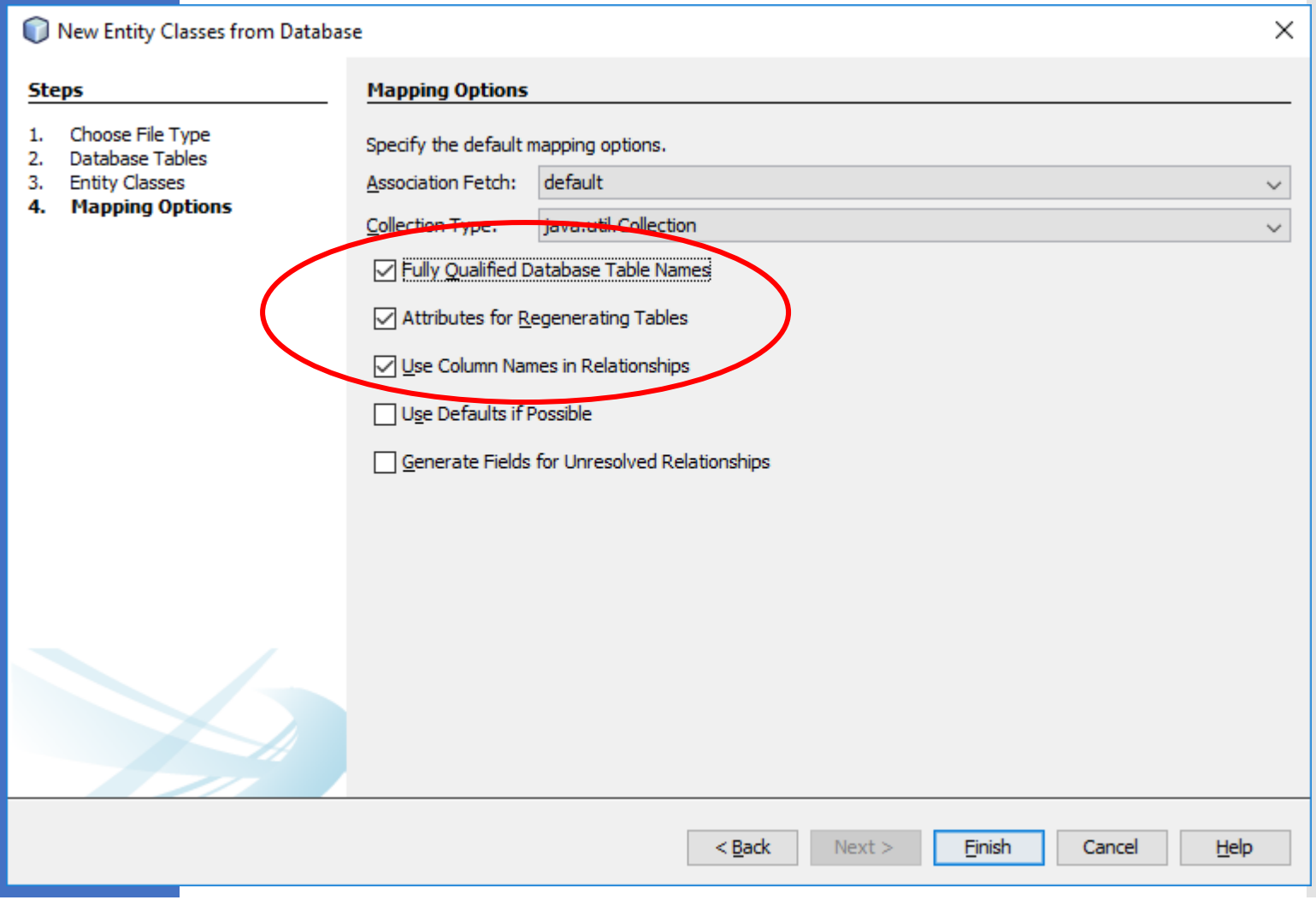
Nota: El servidor debe estar iniciado !!!

3. Generar la entidad

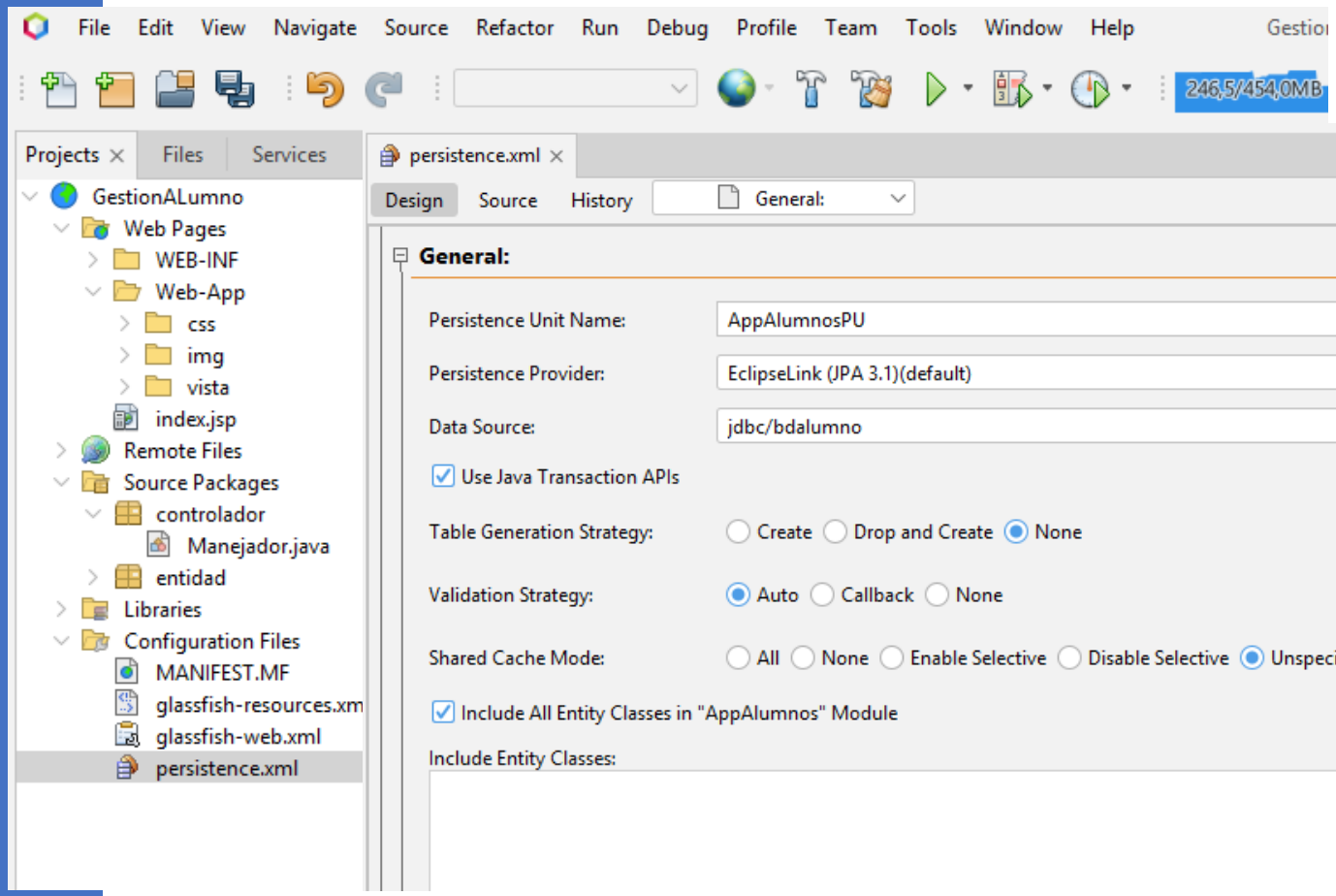


...crear la unidad de persistencia




New Entity Classes from Database
Steps
1. Choose File Type
2. Database Tables
3. Entity Classes
4. Mapping Options
Mapping Options
Specify the default mapping options.
Association Fetch:
default
Collection Type:
java.util.Collection
Fully Qualified Database Table Names
Attributes for Regenerating Tables
Use Column Names in Relationships
Use Defaults if Possible
Generate Fields for Unresolved Relationships
< Back
Next >
Finish
Cancel
Help

//Código Generado -
Unidad de persistencia



Alumno.java

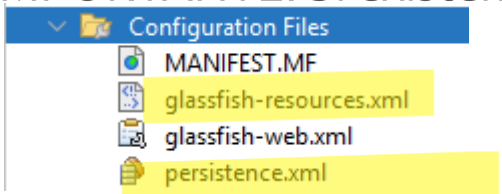
Source | History

```
30  @Entity
31  @Table(name = "alumno", catalog = "bdalumno", schema = "")
32  @XmlRootElement
33  @NamedQueries({
34      @NamedQuery(name = "Alumno.findAll", query = "SELECT a FROM Alumno a"),
35      @NamedQuery(name = "Alumno.findByIdalumno", query = "SELECT a FROM Alumno a WHERE a.idalumno =
36      @NamedQuery(name = "Alumno.findByNombre", query = "SELECT a FROM Alumno a WHERE a.nombre = :nom
37      @NamedQuery(name = "Alumno.findByRegistro", query = "SELECT a FROM Alumno a WHERE a.registro =
    public class Alumno implements Serializable {
39      private static final long serialVersionUID = 1L;
40      @Id
41      @GeneratedValue(strategy = GenerationType.IDENTITY)
42      @Basic(optional = false)
43      @Column(name = "idalumno", nullable = false)
44      private Integer idalumno;
45      @Size(max = 45)
46      @Column(name = "nombre", length = 45)
47      private String nombre;
48      @Size(max = 6)
49      @Column(name = "registro", length = 6)
50      private String registro;
51      @JoinColumn(name = "carrera_idcarrera", referencedColumnName = "idcarrera", nullable = false)
52      @ManyToOne(optional = false)
53      private Carrera carreraIdcarrera;
```

https://en.wikibooks.org/wiki/Java_Persistence/Basic_Attributes

Primera parada

- IMPORTANTE: Si existen, borrar del proyecto:



- Generar las conexiones necesarias para crear las entidades de la base de datos alumnos.



Revisar la clase Manejador

```java
@WebServlet(name = "Manejador",
            loadOnStartup = 1, //Para que el sevlet se  instancia e inicia cdo se depliega
            urlPatterns = {"/SolicitarDatos",
                           "/AgregarAlumno",
                           "/Listar"}
            )

public class Manejador extends HttpServlet {
    @PersistenceUnit
    private EntityManagerFactory emf;
    @Resource
    private UserTransaction utx;

    protected void processRequest(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
            try {
                //response.setContentType("text/html;charset=UTF-8");
                String pathUsuario = request.getServletPath();
                System.out.println("path = "+ pathUsuario);
                String url = null;
                EntityManager em = null;
                switch (pathUsuario) {
                    case "/SolicitarDatos":
                        url = "/WEB-INF/vista/" + pathUsuario + ".jsp";
                        break;
                    case "/AgregarAlumno":
```

```
case "/AgregarAlumno":

    String registro= (String) request.getParameter("registro");
    String nombre  = (String) request.getParameter("nombre");
    String carrera  = (String) request.getParameter("carrera");

    Alumno a = new Alumno();
    a.setRegistro(Integer.parseInt(registro));
    a.setNombre(nombre);
    a.setCarrera(carrera);

    utx.begin();
        em = emf.createEntityManager();
        em.persist(a);
    utx.commit();

    url = "index.jsp";
    break;
case "/Listar":
```

```
        case "/Listar":
            em = emf.createEntityManager();
            Query q = em.createNamedQuery("Alumno.findAll");
            List todos = q.getResultList();
            request.setAttribute("lista",todos);
            url = "/WEB-INF/vista/" + pathUsuario + ".jsp";
            break;
    }

    // usa RequestDispatcher para reTransmitir el requerimiento
    try {
        request.getRequestDispatcher(url).forward(request, response);
    } catch (ServletException | IOException ex) {
    }
```

Recursos

- Que es Apache Derby?
- Anotaciones
- Tutorial Java EE5
- Tutorial Java EE6
- API documents
- Especificaciones Derby